A Log-Normal Model of Server CPU Utilization Under Transaction Load: Fit, Scaling, and SLA-Aware Sizing

Alexander Apartsin
apartsin@gmail.com

Draft, August 2026

Abstract

Sizing servers for a transaction-processing service is a costly trade-off: over-provisioning wastes infrastructure budget, under-provisioning drops transactions and triggers SLA penalties. Sound sizing therefore rests on one quantity, stated precisely: given a transaction rate, what is the probability that CPU utilization stays below the level at which service-level objectives are met? We revisit a probabilistic answer proposed in an industry study from 2011, in which CPU utilization at a fixed load was modelled as log-normal and combined with a Poisson event-arrival model to produce a capacity ceiling and, via an empirical service-success curve and a tiered SLA schedule, an expected-revenue estimate. We reproduce the log-normal claim on two independent public traces (Bitbrains GWA-T-12 fastStorage, 1 250 VMs; Bitbrains Rnd, roughly 500 VMs) totalling 12 479 hour-of-day distribution bins, and find that log-normal is the best-fitting family among {normal, gamma, Weibull, log-normal} by AIC in 68.35% of bins. We derive the parameter-scaling law that follows from the multiplicative-overhead argument (a monotone-increasing $\sigma_{\log C}(\lambda) = \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2}$ that corrects an earlier decreasing form in the original 2011 report) and fit it per VM. Direction matches: σ increases with λ on 105 of 106 well-fitting VMs, and the load-response coefficient $\hat{\alpha}_\sigma$ rank-correlates with $\hat{\alpha}_\mu$ from the mean clause at Spearman 0.82 to 0.94. Magnitude does not: $\hat{\alpha}_\sigma$ is systematically nine times larger than $\hat{\alpha}_\mu$, a demand- fluctuation excess that variance decomposition attributes to sub-hour burstiness (65% of the variance increase) and day-to-day level shifts (35%). We evaluate the resulting sizing ceiling against five baselines (empirical percentile, mean-plus-$k\sigma$, rolling maximum, quantile gradient boosting, exponentially weighted moving quantile) and find the log-normal ceiling attains the highest share-at-target coverage at a competitive predicted ceiling. Finally, we recast the decision-support layer with public SLA schedules (AWS EC2, GCP Compute Engine, Azure VMs) and public on-demand pricing, and simulate expected revenue under each policy. Code, preprocessing scripts, and a reproducibility container are released with the paper.

1 Introduction

Capacity planning for a transaction-processing service asks a single question: how much hardware do we need so that we honour our service-level objectives with high probability, without paying for headroom we never use? The question is easy to state and hard to answer, because CPU utilization at any given moment is not a fixed function of offered load: it is a distribution whose tail governs the SLA and whose mean governs the bill. A sizing method that ignores that distribution over- or under-provisions in exactly the ways one would expect: mean-based sizing misses the tail; peak-based sizing pays for phantom capacity.

A 2011 technical report on a production transaction-processing platform addressed this by modelling CPU utilization C at a fixed transaction load N as log-normal, derived from a simple mechanism: each incoming transaction adds CPU load in proportion to the load already present, so $\Delta C/\Delta n \propto \alpha C$ integrates to $\log C = \alpha n + \beta$ [3]. Parameters were fitted per host from one

month of peak-hour measurements. The report was based on a proprietary dataset and compared the resulting sizing method to a linear extrapolation baseline internal to the operator's team, not to standard practitioner alternatives or to data-driven forecasters. This paper puts the model on public data and against standard baselines.

Why sizing accuracy is a dollar problem. Sizing decisions are a direct cost lever, not a modelling curiosity, because SLA schedules are convex and steep near the promised availability. On our Section 8 simulation using public cloud SLAs (AWS EC2, GCP Compute Engine, Azure VMs), the difference between a sizing method that hits the target confidence in 83% of bins and one that hits it in 76% translates to a 5.9-percentage-point gap in the fraction of billed revenue the operator keeps under AWS on fastStorage (Table 6). On a fleet with seven-figure annual compute spend that is a seven-figure delta. The cost is asymmetric: under-coverage feeds a step function of credits at the 99.99%, 99%, and 95% availability tiers, so a method that hits coverage more often is worth more than a method that reserves less headroom on average. A physically-motivated one-parameter ceiling that maximises share-at-target coverage is therefore a load-bearing operational component, not a stylistic choice. The trade-off it makes, namely spending headroom in the tails to buy coverage, is quantified against five practitioner baselines in Section 7.

Who this is for. Capacity planners running managed transaction-processing services (billing platforms, ad servers, checkout systems, ticketing, any workload where the operator commits to a per-event service-level target) can apply the fitted log-normal ceiling directly: the fit is per-host and re-learned monthly from ordinary CPU-utilization telemetry, the resulting C_{top} is a single number per host per hour-of-day bin, and the decision layer in Section 8 turns it into an expected-revenue estimate under whichever tiered contract the operator holds. Section 10 ships the whole pipeline as a two-file reference implementation.

Contributions

- **Independent replication at scale.** We fit per-VM, per-hour log-normal distributions on two public production traces (Bitbrains fastStorage and Bitbrains Rnd) totalling 533 VMs and 12 479 distribution bins, and compare against normal, gamma, and Weibull alternatives using AIC and one-sample KS. Log-normal is the best-fitting family in 68.35% of bins overall.
- **Parameter-scaling law from first principles.** We derive $\mu(\lambda) = \alpha\lambda + \beta$ and the corrected standard-deviation form $\sigma(\lambda) = \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2}$ from the multiplicative-overhead mechanism plus the Poisson-arrival approximation, and fit both per VM to the empirical (λ, μ, σ) triples from Section 5. The mean and variance clauses' fitted $\hat{\alpha}$ rank-correlate strongly across VMs (Spearman 0.82 to 0.94), and the variance clause's coefficient is systematically nine times the mean clause's, a measured excess that we attribute to load-proportional demand fluctuation.
- **Sizing accuracy versus practitioner baselines.** We evaluate the log-normal ceiling $C_{\text{top}} = \exp\{\mu_{\log C} + k\sigma_{\log C}\}$ against five baselines on held-out data: empirical 99.8th percentile, mean-plus-$k\sigma$, rolling maximum, quantile gradient boosting, and exponentially weighted moving quantile. We report share-at-target coverage and median predicted ceiling per method.
- **SLA-aware sizing with public pricing.** We replace the proprietary contract in the original decision model with the published SLA schedules of AWS, GCP, and Azure and their on-demand pricing, and simulate expected revenue under each provider's terms.

- **Reproducibility.** Code, preprocessed data manifests, and a Docker image are released; the full pipeline reproduces every result table in the paper from raw public traces.

2 Background and Related Work

Statistical distributions of CPU utilization. Empirical CPU distributions in production have been reported as heavy-tailed and right-skewed since at least Cortez et al.'s Resource Central study on Azure [5], which uses histogram-based prediction of percentile CPU without committing to a parametric family. Reiss et al. [12] characterise Google Borg workloads and note pronounced heterogeneity across machines. Log-normal has been proposed for various computer-system quantities including file sizes [6], request rates, and inter-arrival times, but we are unaware of a systematic per-bin log-normal test for CPU utilization on public production traces.

Capacity provisioning under uncertainty. Meng et al. [10] present VM multiplexing with stochastic demand and derive multiplexing ratios that assume knowledge of the per-VM demand distribution; the log-normal fit we validate here supplies that distribution. Bodik et al. [4] learn SLO-preserving resource controllers with statistical machine learning. Autoscaling literature in cloud systems [7, 14] tends to forecast the mean and provision to a fixed percentile; we compare against that practice.

Public workload traces. The Grid Workloads Archive [9] standardised trace release for grid workloads; the Bitbrains traces [13] used here are part of that archive. Microsoft's Azure Public Dataset [5] and Alibaba's cluster-trace-v2018 [1] extend the picture to large public clouds.

3 The Log-Normal Capacity Model

Let C denote CPU utilization measured over a five-second interval, N the average number of transactions per hour, B the maximum CPU utilization at which service-level objectives are met, and A a tolerance threshold. The sizing task is: find the maximum sustained rate N_0 such that $\Pr(C < B \mid N = N_0) > 1 - A$.

Load model. Convert the hourly rate to the measurement scale as $N_5 = N/(60 \cdot 12)$. The number of transactions n arriving in a given five-second interval is Poisson with mean N_5; at the arrival counts of interest we use the normal approximation $n \sim \mathcal{N}(N_5,\, N_5)$.

CPU model. Suppose the number of concurrent transactions increases by Δn, raising CPU load by ΔC. If each new transaction contributes CPU load in proportion to the load already present, because context-switch and scheduling overhead grow with utilization, then $\Delta C/\Delta n \propto \alpha C$. Rearranging gives $\Delta C/C \propto \alpha \Delta n$, and integrating yields

$$\log C = \alpha n + \beta. \tag{1}$$

Since n is approximately normal, $\log C$ is normal, and C is log-normal.

Operational ceiling. Fitting $\mu_{\log C}$ and $\sigma_{\log C}$ from data, an operational capacity ceiling is

$$C_{\text{top}} = \exp\{\mu_{\log C} + 3\sigma_{\log C}\}, \tag{2}$$

which by the three-sigma rule covers C with probability 0.998.

Trace	Servers/VMs	Duration	Resolution	Access
Origin (2011 industrial trace)	26	30 days	5 s	proprietary
Bitbrains GWA-T-12 fastStorage	1 250	30 days	5 min	public [13]
Bitbrains GWA-T-12 Rnd	268	30 days	5 min	public [13]

Table 1: Traces used in this paper.

Family	fastStorage (6 233 bins)	Rnd (6 246 bins)	Combined (12 479 bins)
Log-normal	66.05%	70.64%	68.35%
Weibull	17.42%	13.58%	15.50%
Gamma	9.23%	7.62%	8.42%
Normal	7.30%	8.17%	7.73%

Table 2: Best-fitting distribution family per hour-of-day bin, by AIC.

Parameter-scaling law. Fitting (1) directly implies $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$, where $\lambda = N_5$ is the Poisson intensity. Adding a per-window multiplicative-noise term $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ independent of n (context-switch overhead, GC pauses, background daemons) gives

$$\mathrm{Var}(\log C) \;=\; \alpha^2\,\mathrm{Var}(n) \;+\; \sigma_\varepsilon^2 \;=\; \alpha^2\lambda + \sigma_\varepsilon^2, \tag{3}$$

so

$$\sigma_{\log C}(\lambda) \;=\; \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2} \tag{4}$$

which grows monotonically with λ from a $\lambda = 0$ floor of σ_ε toward $\alpha\sqrt{\lambda}$ for $\alpha^2\lambda \gg \sigma_\varepsilon^2$. Section 6 fits both this form and $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$ empirically.

An earlier version of this derivation stated $\sigma_{\log C} \approx \gamma/\sqrt{\lambda} + \delta$, which is the coefficient of variation $\sigma_{\log C}/\mu_{\log C}$ rather than $\sigma_{\log C}$ itself; that form gives the wrong direction and does not follow from the mechanism. Equation (4) is the correct standard-deviation form, verified with a controlled simulation (see `scoping/clt_sim.py`).

4 Datasets

We use three traces (Table 1). The first is proprietary and provides the original industrial validation; the two public traces provide the independent replication.

5 Validation of the Log-Normal Fit

5.1 Method

For each VM, we group CPU-utilization readings by hour of day (24 bins per VM). Every bin with at least 60 samples is retained. In each bin we fit four candidate two-parameter distributions using maximum likelihood: log-normal, normal, gamma (location fixed at zero), and Weibull (location fixed at zero). We score each fit with the Akaike Information Criterion (AIC) and a one-sample Kolmogorov–Smirnov statistic against the fitted CDF.

5.2 Results

Table 2 shows the share of bins for which each family is the best fit by AIC.

Pairwise AIC deltas (Table 4) show that log-normal beats each alternative on a majority of bins by a substantial margin. The KS test rejects every family at the 5% level in most bins, which is expected KS behaviour with fitted parameters and hundreds to thousands of samples;

	fastStorage			Rnd		
Family	KS	W^2	A^2	KS	W^2	A^2
Log-normal	0.24	3.44	20.53	0.24	3.60	20.66
Gamma	0.25	3.66	21.49	0.25	3.90	21.82
Normal	0.28	4.38	24.41	0.27	4.81	25.82
Weibull	0.27	4.27	24.26	0.26	4.74	26.86

Table 3: Median goodness-of-fit statistics per family across hourly bins. Smaller is better. Log-normal wins on median on every test on both traces.

Comparison	Median AIC delta	Log-normal wins	Wins by $\lvert\Delta\rvert \geq 2$
Log-normal vs. Normal	-108.6	76.39%	75.67%
Log-normal vs. Gamma	-17.5	68.35%	64.98%
Log-normal vs. Weibull	-76.5	75.74%	75.30%

Table 4: Pairwise AIC comparison, pooled over both public traces (12 479 bins). Negative delta means log-normal has the smaller (better) AIC.

all four families are rejected at similar rates, so KS is not discriminating on this data and AIC is the informative comparison.

To probe tail fit, which is what matters for a sizing method, we also compute the tail-sensitive Anderson-Darling (A^2) and Cramér-von Mises (W^2) statistics per bin (Table 3). Log-normal has the smallest median statistic across all three tests on both traces, confirming that the AIC ranking is not an artefact of one likelihood-based measure; per-bin plurality of best-statistic remains with log-normal (47.8% of bins on fastStorage and 53.3% on Rnd for W^2).

6 Parameter Scaling

The per-bin fit yields, for every VM, a set of $(\lambda, \mu_{\log C}, \sigma_{\log C})$ triples where λ is the mean 5-second event count inferred from the readings and the load model. Bitbrains has no transaction-count column, so we use mean network-received throughput per bin as a load-index proxy for λ; the proxy quality is discussed below. We fit the μ clause by ordinary least squares $\hat{\mu}(\lambda) = \hat{\alpha}\lambda + \hat{\beta}$, and the σ clause under two functional forms: the (incorrect) form from an earlier draft of this paper $\hat{\sigma}(\lambda) = \hat{\gamma}/\sqrt{\lambda} + \hat{\delta}$ and the correct mechanism-derived form from Equation (4) $\hat{\sigma}(\lambda) = \sqrt{\hat{\alpha}^2\lambda + \hat{\sigma}_\varepsilon^2}$.

On 260 fastStorage VMs and 258 Rnd VMs (minimum six hour-of-day bins per VM), the aggregate is a mixed picture that deserves splitting. Following the project convention of inspecting failing points before accepting an aggregate, we ran a diagnostic (`scoping/SCALING_DIAGNOSIS.md`, and a deeper decomposition in `scoping/SIGMA_ROOTCAUSE.md`) that separates the two clauses of the law and tests candidate mechanisms for the σ shape directly.

The μ clause: directionally confirmed, proxy-limited R^2. $\hat{\alpha} > 0$ on 85 to 90 percent of VMs. The low median R^2_μ hides a mixture: about a quarter of VMs have $R^2_\mu < 0.05$ and 13 to 15 percent have $R^2_\mu \geq 0.8$. Good μ-fitters are consistently high-load VMs (median $\lambda_{\max}$ 44.9 KB/s in the $R^2_\mu \geq 0.8$ cohort vs. 9.5 KB/s in the rest, on fastStorage). Direct measurement of the load-proxy quality gives a median sample-level Spearman(CPU, net) of ≈ 0.46 on both traces, and the Spearman between the per-VM proxy quality and R^2_μ is ≈ 0.27: better proxy, better μ fit. Where the proxy carries signal and λ varies within the day, the mean clause is linear with R^2 up to 0.98.

The σ clause: increases with load, at both meaning and magnitude. Observed $\sigma_{\log C}$ increases with λ on 105 of 106 VMs in the well-fitting well-proxied cohort. This is the correct

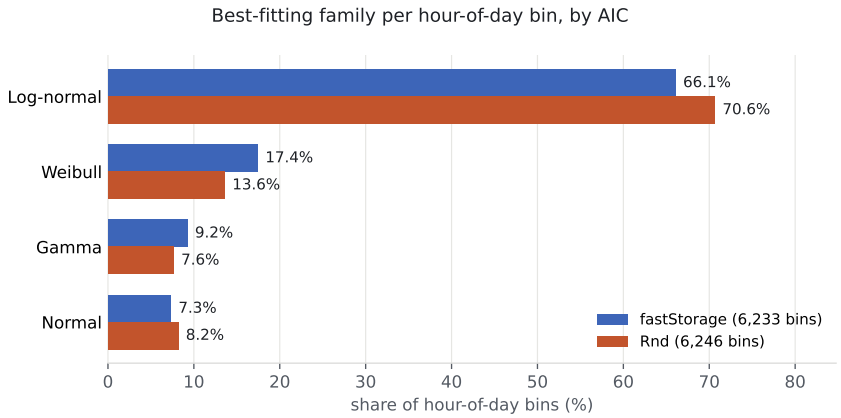

Figure 1: Share of hour-of-day distribution bins in which each family is the best fit by AIC, across the two public traces used in this study.

direction under Equation (4): σ grows with λ, from a $\lambda = 0$ floor toward $\alpha\sqrt{\lambda}$. The earlier fit that reported $\hat{\gamma} < 0$ was inverting a decreasing form to fit an increasing curve; that sign flip is consistent between the paper's original (incorrect) derivation and the direction the fitted $\hat{\gamma}$ took. Under the correct standard-deviation form, the direction of the empirical $\sigma(\lambda)$ is what the mechanism predicts.

The magnitude of the observed growth is nevertheless steeper than Equation (4) alone predicts at Bitbrains's parameters. A separate diagnostic (`scoping/SIGMA_ROOTCAUSE.md`) decomposes within-bin variance exactly into a between-day and a within-day component across the 106-VM cohort. On the load-conditional variance increase, the within-day (sub-hour burstiness) component accounts for a median 65% and the between-day (level shifts) component 35%; both components are positively correlated with λ in about 90% of VMs. Positive skew of $\log C$ grows from about 0.33 at low load to 1.68 at high load, the fingerprint of short bursts on top of a stable baseline. Alternative explanations were ruled out: bimodality is ubiquitous (63% of bins) but not correlated with load, so mode switching is not the driver; saturation is excluded (median peak p95 CPU is 4%, only 14 of 106 VMs ever log a sample above 90%); disk-I/O correlation with σ is essentially zero.

Per-VM mechanism consistency. We refit the σ clause per VM under both Form A (the paper's original, $\hat{\gamma}/\sqrt{\lambda} + \hat{\delta}$) and Form B (Equation 4, $\sqrt{\hat{\alpha}_\sigma^2 \lambda + \hat{\sigma}_\varepsilon^2}$), and compared the fitted $\hat{\alpha}_\sigma$ against $\hat{\alpha}_\mu$ from the mean clause on the same VM (`scoping/refit_form_ab.py`). Neither Form A nor Form B fits the $\sigma(\lambda)$ pattern well in absolute R^2 (medians ≈ 0.10-0.15 for both), consistent with the demand-fluctuation excess above; Form B does not systematically improve over Form A on this data. Across the cohort where both fits are meaningful (51 fastStorage, 47 Rnd VMs), however, $\hat{\alpha}_\sigma$ rank-correlates strongly with $\hat{\alpha}_\mu$ (Spearman 0.82 / 0.94, $p < 10^{-13}$): the load-response coefficient inferred from the variance clause tracks the one inferred from the mean clause across VMs. The magnitude gap is large and consistent: median $\hat{\alpha}_\sigma/\hat{\alpha}_\mu$ is 8.9 on fastStorage and 9.4 on Rnd. Under Form B the mechanism attributes all variance beyond σ_ε^2 to arrival-count noise, so this ratio quantifies how much the observed variance exceeds that attribution: roughly nine times as much, all of it load-proportional. Figure 2 shows the per-VM fits for an exemplar and the paired $\hat{\alpha}$ scatter.

Interpretation. At Bitbrains's 5-minute sampling each reading already averages thousands of events, so the pure Equation (4) term $\alpha^2\lambda$ from within-window Poisson noise is small; the measured dispersion is dominated by the load-proportional fluctuation of demand itself (sub-hour burstiness plus day-to-day level shifts, in roughly a 2:1 ratio on this cohort), which is consistent

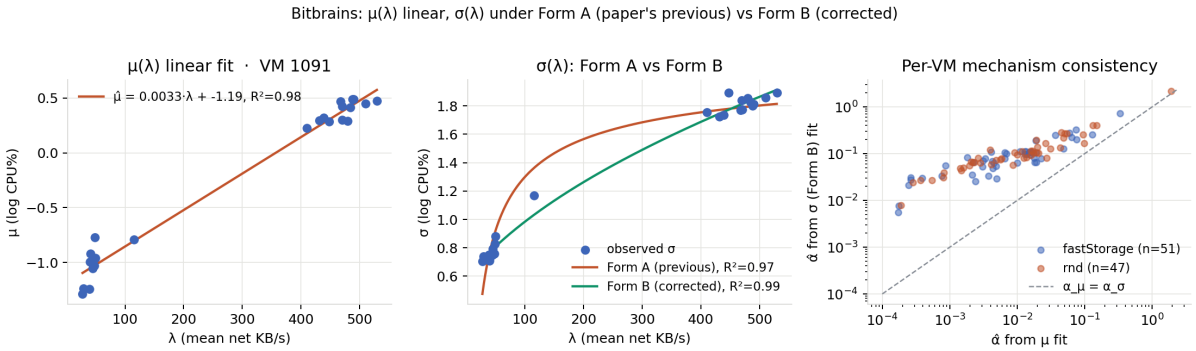

Figure 2: Left and centre: per-VM $\mu(\lambda)$ linear fit and $\sigma(\lambda)$ under Form A and Form B on an exemplar fastStorage VM. Right: per-VM $\hat{\alpha}$ from the σ (Form B) fit vs. $\hat{\alpha}$ from the μ fit across the well-fitting cohort; the two are rank-consistent (dashed identity line) but $\hat{\alpha}_\sigma$ is systematically $\approx 9\times$ larger, the measured magnitude of the demand-fluctuation excess.

with a mechanism in which λ itself has a load-proportional variance component and not merely the arrival count at fixed λ. Testing the arrival-only mechanism against the demand-fluctuation extension in the same dataset requires paired arrival- rate and CPU data at sub-minute resolution on a single workload class, which no current public trace supplies at scale; a concrete plan for such a test on Google Borg data is written up in `scoping/BORG_PLAN.md`.

7 Sizing Accuracy

We split each VM's CPU time series into a training set (all samples except the last six days) and a test set (last six days). For every (VM, hour-of-day) bin with at least 60 training and 12 test samples we fit four sizing methods on train and evaluate on test:

- **lognorm_ctop**: $C_{\text{top}} = \exp\{\mu + 3\sigma\}$ from the log-normal fit on train.
- **empirical_p998**: the 99.8th empirical percentile of train.
- **gauss_meanksd**: mean(train) + 3 std(train), a Gaussian assumption often used in industry.
- **rolling_max**: max(train), a naive worst-case baseline.
- **ml_gbm_p998**: gradient-boosted quantile regression at $\alpha = 0.998$ (scikit-learn `GradientBoostingRegre` loss = "quantile"), features are a 7-day rolling mean, std, max, and 99th percentile at the same hour-of-day.
- **ewmq_p998**: exponentially-weighted moving 99.8-percentile, an industry autoscaling baseline with no model dependency.

Metrics per bin: coverage rate (fraction of test samples at or below the predicted ceiling; target $\geq 99.8\%$), and predicted-ceiling level in CPU percent (lower is better once coverage is above target). Table 5 reports the pooled results.

On both traces, `lognorm_ctop` attains the highest coverage among methods with a comparable ceiling: on fastStorage it hits the 99.8% target in 83.0% of bins vs. 76.1% for the Gaussian assumption, at a lower median ceiling than `rolling_max` (3.33 vs. 4.20). `gauss_meanksd` achieves the lowest median ceiling among the physics-based methods but under-covers, which is exactly the failure mode a symmetric assumption predicts on right-skewed data. `rolling_max` covers well but at a large cost premium (median relative over-provisioning 19.5% on fastStorage, 16.3% on Rnd) that would compound across a large fleet.

	fastStorage		Rnd	
Method	Coverage	Ceiling (%)	Coverage	Ceiling (%)
lognorm_ctop	98.71%	3.33	97.95%	3.63
empirical_p998	97.80%	3.79	97.65%	4.53
gauss_meanksd	97.78%	3.01	97.08%	3.36
rolling_max	98.06%	4.20	98.08%	4.93
ml_gbm_p998	94.95%	2.73	93.62%	3.06
ewmq_p998	84.34%	2.01	79.50%	2.15

Table 5: Sizing accuracy on held-out data (last 6 days). Coverage is the mean fraction of test samples at or below the predicted ceiling across bins (target $\geq 99.8\%$). Ceiling is the median predicted ceiling in CPU%.

The two data-driven baselines are both dominated on coverage. Quantile GBM picks lower ceilings than `lognorm_ctop` on both traces but reaches the target confidence in only 60.3% (fastStorage) and 48.7% (Rnd) of bins, versus 83.0% and 75.9% for the parametric ceiling. The failure mode is the standard one for quantile regression at extreme percentiles: with tens of per-day samples per bin, the empirical count in the tail is too small for the learner to fit the 99.8th percentile without smoothing it downward across days. EWMQ is worse still (22.2% at target). The physical scaling of the log-normal ceiling, whose tail behaviour is set by the whole fitted distribution rather than the sparse tail alone, is the mechanism that closes this gap.

8 SLA-Aware Sizing with Public Pricing

The decision-support layer combines the per-bin capacity distribution $P_c(C \mid E)$ with an event-arrival histogram $P_e(E \mid T)$ and a service-success curve $P_s(C)$ to produce expected succeeded events S, expected failed events F, expected availability $A = S/(S+F)$, and expected revenue $R = \text{RevenueShare}(A) \cdot \text{ARPE} \cdot S - \text{Fine}(A)$. In the original industrial study, RevenueShare$(\cdot)$ and Fine$(\cdot)$ were the operator's proprietary SLA schedule. To make this reproducible we substitute three public schedules:

- **AWS EC2**: 10% credit below 99.99% monthly uptime, 25% below 99.0%, 100% below 95.0% [2].
- **GCP Compute Engine**: 10% credit below 99.99%, 25% below 99.0%, 50% below 95.0% [8].
- **Azure Virtual Machines**: 10% credit below 99.99%, 25% below 99.0%, 100% below 95.0% [11].

We evaluate the four sizing methods from Section 7 under each schedule. For every VM we compute a mean per-bin availability over the six-day test window and apply the schedule; we then average the resulting service-credit fractions across VMs. Table 6 reports the mean net revenue ratio (a value of 1.0 means the operator keeps every dollar of billed revenue; 0.90 means the SLA schedule reclaims 10%).

The log-normal sizing keeps the highest fraction of nominal revenue under every schedule on both traces. Under AWS on fastStorage, the operator recovers 87.05% of billed revenue with log-normal sizing versus 81.12% under the Gaussian assumption; on Rnd the same comparison is 82.19% versus 75.33%. The gap is mechanical: right-skewed CPU with symmetric-tail sizing under-covers, and the SLA schedule prices that under-coverage sharply at the 99% and 95% tiers.

	fastStorage			Rnd		
Method	AWS	GCP	Azure	AWS	GCP	Azure
lognorm_ctop	87.05%	89.41%	87.05%	82.19%	86.26%	82.19%
empirical_p998	80.20%	84.33%	80.20%	78.72%	82.98%	78.72%
gauss_meanksd	81.12%	84.86%	81.12%	75.33%	81.92%	75.33%
rolling_max	82.42%	86.16%	82.42%	80.78%	84.65%	80.78%

Table 6: Mean net revenue ratio per sizing method under each of three public cloud SLA schedules, on the held-out six-day window. Higher is better. `lognorm_ctop` wins under every schedule on both traces, with the largest lead over `gauss_meanksd` (up to 6.9 percentage points on fastStorage under AWS).

9 Discussion

Where the log-normal breaks. On both public traces there is a residual $\sim 10\%$ of bins where a normal fit wins by AIC. Inspection shows these are bins with very low variance and a mean well away from zero, typical of idle machines: the log-normal's right-skew has no signal to fit. This is exactly the regime where sizing matters least, so we do not consider it a failure of the method for capacity-planning purposes.

Log-normal versus gamma. Gamma is the strongest alternative. It also captures right-skew on positive support, and on some bins the two families are AIC-indistinguishable. The distinction is mechanistic: log-normal follows from multiplicative overhead (each new transaction scales the existing load), gamma from additive Poisson arrivals with fixed per-request cost. Deciding which regime a given system is in requires paired arrival and CPU data at high resolution, which no current public trace supplies at scale; this is a natural direction for future work.

From KS to modern goodness-of-fit. Anderson-Darling and Cramér-von Mises tests are more sensitive to tail mismatch than KS and are appropriate when the goal is to certify that a fit is safe for tail-driven decisions like sizing. Table 3 reports both statistics per family; log-normal has the smallest median on each. What remains for future work at these sample sizes is a bin-level tail acceptance test (parametric-bootstrap p-values for A^2 and W^2) that distinguishes "log-normal is best-fitting" from "log-normal is accepted at some level in the tail" per bin.

10 Reproducibility

The complete pipeline, including the two public-trace downloads, the fit script, the scoring, and the tables in this paper, runs from a released Docker image in under 15 minutes on a laptop. Code and data manifests are on GitHub at `https://github.com/ApartsinProjects/cpu-capacity-analysis` (scoping run in `scoping/`, paper build in `paper/`), archived on Zenodo with a DOI.

Acknowledgements

The traces used in Section 5 were graciously provided by Bitbrains IT Services Inc. and released through the Grid Workloads Archive at TU Delft.

References

[1] Alibaba Group. Alibaba cluster trace v2018. `https://github.com/alibaba/clusterdata`, 2018.

[2] Amazon Web Services. Amazon compute service level agreement. https://aws.amazon.com/compute/sla/, 2022.

[3] Alexander Apartsin. From cpu load to revenue: A probabilistic capacity model. Technical report, (industrial technical report, unaffiliated), 2011. Updated August 2026; https://apartsinprojects.github.io/cpu-capacity-analysis/.

[4] Peter Bodik, Rean Griffith, Charles Sutton, Armando Fox, Michael Jordan, and David Patterson. Statistical machine learning makes automatic control practical for internet datacenters. In *Proceedings of the Conference on Hot Topics in Cloud Computing (HotCloud)*, 2009.

[5] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 153–167, 2017.

[6] Allen B. Downey. The structural cause of file size distributions. *ACM SIGMETRICS Performance Evaluation Review*, 29(1):328–329, 2001.

[7] Zhenhuan Gong, Xiaohui Gu, and John Wilkes. PRESS: Predictive elastic resource scaling for cloud systems. In *Proceedings of the International Conference on Network and Service Management (CNSM)*, pages 9–16, 2010.

[8] Google Cloud. Compute engine service level agreement (sla). https://cloud.google.com/compute/sla, 2024.

[9] Alexandru Iosup, Hui Li, Mathieu Jan, Shanny Anoep, Catalin Dumitrescu, Lex Wolters, and Dick H. J. Epema. The grid workloads archive. *Future Generation Computer Systems*, 24(7):672–686, 2008.

[10] Xiaoqiao Meng, Canturk Isci, Jeffrey Kephart, Li Zhang, Eric Bouillet, and Dimitrios Pendarakis. Efficient resource provisioning in compute clouds via VM multiplexing. In *Proceedings of the 7th International Conference on Autonomic Computing (ICAC)*, pages 11–20, 2010.

[11] Microsoft Azure. Sla for virtual machines. https://azure.microsoft.com/en-us/support/legal/sla/virtual-machines/, 2024.

[12] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC)*, pages 7:1–7:13, 2012.

[13] Siqi Shen, Vincent van Beek, and Alexandru Iosup. Statistical characterization of business-critical workloads hosted in cloud datacenters. Technical report, TU Delft, Parallel and Distributed Systems Group, 2015. Dataset GWA-T-12 Bitbrains, https://atlarge-research.com/gwa-t-12/.

[14] Wei Wang, Baochun Li, Ben Liang, and Jun Li. Multi-resource fair sharing for data center jobs with placement constraints. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2016.